Step 4 – Wrap up

We presented the architecture design for video streaming services like YouTube. If there is extra time at the end of the interview, here are a few additional points:

1. Scale the API Tier

- API servers are stateless, so horizontal scaling is easy.
- Add more API servers behind a load balancer to handle increasing traffic.
- Frequently accessed data can be cached using Redis or Memcached to reduce database load.

2. Scale the Database

- Use **database replication** for high availability and read scalability.
- Use **database sharding** to distribute data across multiple database servers.
- Separate read and write traffic using master-slave architecture.

3. Live Streaming

Live streaming refers to recording and broadcasting videos in real time.

Although our system is mainly designed for non-live streaming, both systems share similarities:

- Video uploading
- Video encoding/transcoding
- Video streaming through CDN

Differences between live and non-live streaming:

- **Lower latency requirement**
    - Live streaming requires extremely low delay.
    - Different streaming protocols may be needed.
- **Lower parallelism**
    - Small chunks are processed continuously in real time.
    - No need to wait for full video upload.
- **Different error handling**
    - Long retry operations are unacceptable.
    - Fast failover and quick recovery are essential.

4. Video Takedowns

The system should support removing videos that violate policies such as:

- Copyright infringement
- Pornographic content
- Violent or illegal content

Videos can be detected in two ways:

1. Automatically during upload using content scanning systems.
2. Through user reports and moderation workflows.